Projet Numérique.

Projet niveau A : Génétique.

Interpolation de données expérimentales d'un plasma par une fonction Lorentzienne.

Groupe de TP : A32

Noms / Prénoms :

RAMMAL Ziad

XU Jinhao

WEI xing

Date : 16/12/2023

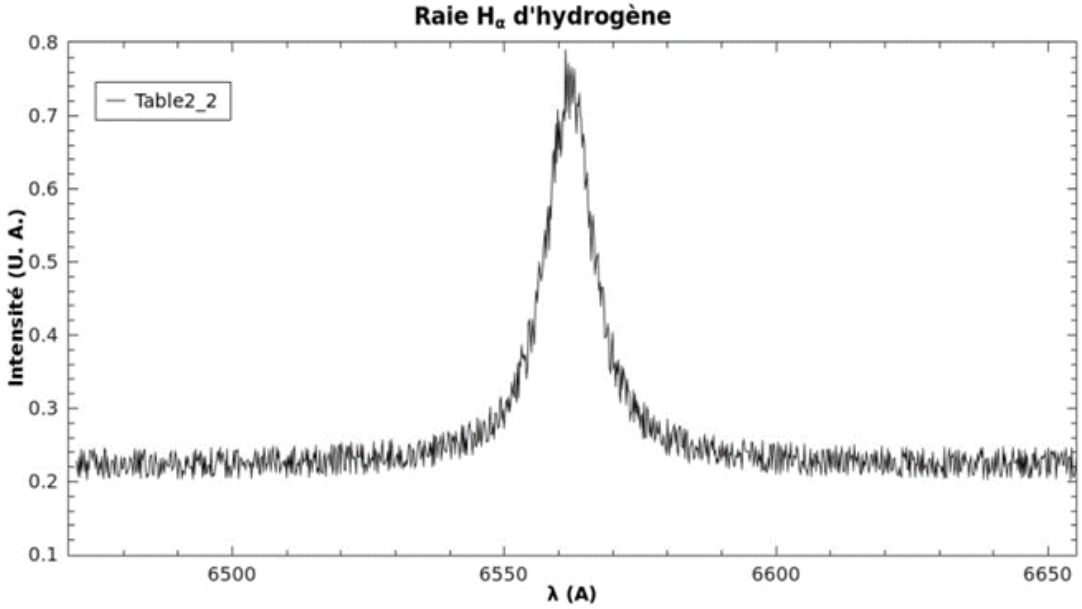


UNIVERSITÉ TOULOUSE III PAUL SABATIER



Présentation du sujet :

En cas de défaut sur un réseau électrique domestique, un disjoncteur assure la coupure du courant en utilisant un arc électrique généré entre deux électrodes. Ce dispositif comprend une chambre à arc avec des plaques métalliques à son extrémité. Lors d'un défaut, les électrodes s'ouvrent, créant un arc électrique qui persiste. Les industriels cherchent à optimiser ce dispositif en comprenant mieux les caractéristiques de l'arc électrique. Une approche consiste à analyser spectaculairement la lumière émise par l'arc, en particulier la raie Hα de l'hydrogène. En mesurant cette raie à travers la spectroscopie, notamment en présence d'humidité, on peut obtenir des informations sur la densité électronique du plasma, influant sur sa capacité à conduire l'électricité. La figure 1 illustre un exemple de mesure.



L’objectif de notre projet est de trouver les meilleures paramètres d’une fonction Lorentzienne : Y0, B, λ0, ΔλL

La fonction est de type :

$$J(\lambda) = Y_0 + B\frac{\text{i}}{(\lambda-\lambda_0)^2+\left(\frac{\Delta\lambda_L}{2}\right)^2}$$

Présentation de méthode numérique :

La méthode numérique utilisée dans ce projet est les algorithmes génétiques binaires, une forme d'algorithme génétique (GA) qui est principalement utilisée pour résoudre des problèmes d'optimisation. L'idée de base de cet algorithme est d'imiter les mécanismes évolutifs de la sélection naturelle et de la génétique. Dans les algorithmes génétiques binaires, la solution est souvent représentée comme une chaîne de nombres binaires (0 et 1), qui est similaire au codage des chromosomes dans la nature.

L'algorithme génétique binaire est divisé en plusieurs étapes : initialisation de la population, fonction d'évaluation de la santé physique des individus, sélection des individus, croisement, discussion de la situation de mutation, production de la prochaine génération de population et conditions de fin.

Initialisation : une population initiale est générée de façon aléatoire. Chaque individu est composé d'une chaîne de nombres binaires.

Évaluation de la condition physique : la condition physique de chaque individu est évaluée selon une fonction de condition physique prédéfinie.

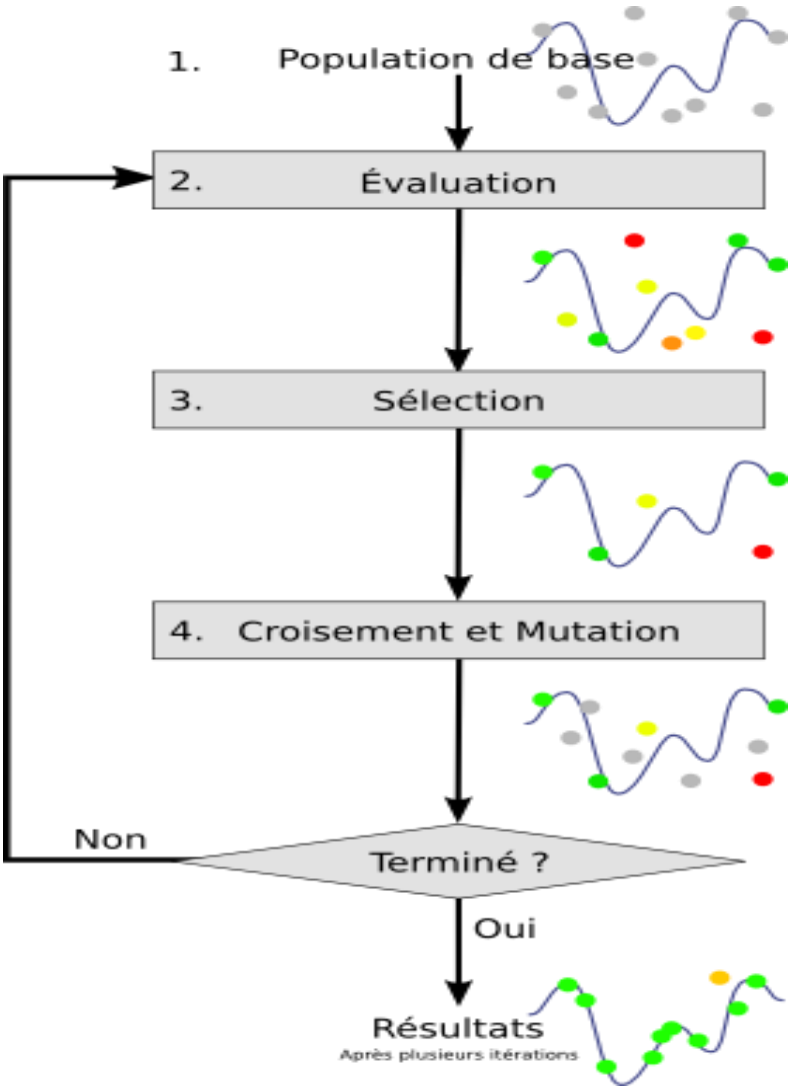
Sélection : sélection basée sur la condition physique de l'individu. Les individus sont généralement sélectionnés sur la base de leur aptitude, et les individus ayant une aptitude élevée ont une plus grande probabilité d'être sélectionnés pour participer à la reproduction de la prochaine génération.

Croisement : les individus sélectionnés se croisent pour produire une nouvelle descendance. Dans les algorithmes génétiques binaires, cela implique généralement que deux chromosomes échangent une partie de leurs gènes dans des positions aléatoires.

Mutation : certains gènes dans les chromosomes sont modifiés de façon aléatoire (par exemple, en changeant 0 à 1 ou 1 à 0) afin de maintenir la diversité de la population.

Population de nouvelle génération : de nouveaux individus issus de la sélection, du croisement et de la variation forment une population de nouvelle génération. Le processus ci-dessus est ensuite répété.

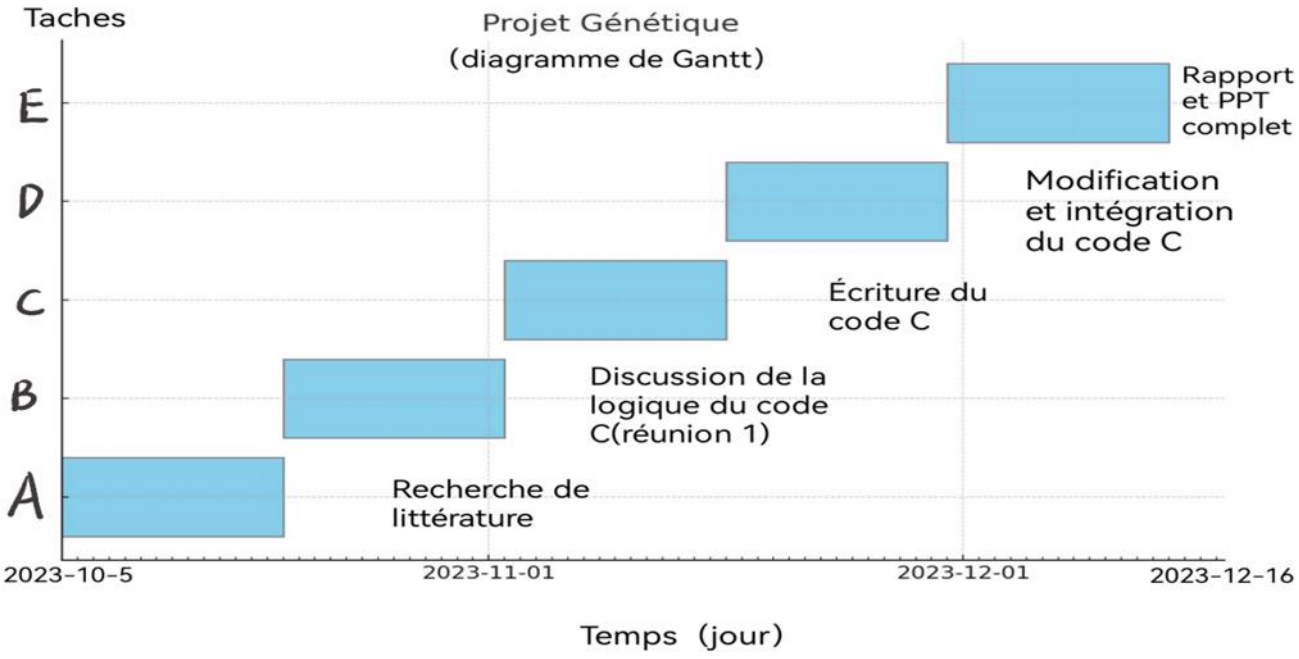
Condition de fin : le processus est répété jusqu'à ce qu'une certaine condition prédéterminée soit atteinte, comme avoir exécuté une algèbre fixe, ou avoir trouvé une solution suffisamment bonne.



Gestion de projet :

Fiches de tache :

A. Recherche de littérature	Lire les références sur les algorithmes génétiques binaires et recueillir des informations pertinentes
B. Discuter de la logique du code C	Sur la base des informations acquises, organiser des réunions de groupe pour partager et discuter des idées logiques et opérationnelles sur le code C de l'algorithme génétique binaire
C. Écriture du code C	Répartition du travail en fonction de la pensée et de la logique d'écriture, chacun étant responsable de l'écriture d'une partie du code C
D. Modification et intégration du code C	Mener une deuxième Réunion du groupe, intégrer les différentes parties de l'écriture du code C terminé, exécuter et découvrir les problèmes, ensemble pour modifier
E. Achèvement du Rapport et PPT	Compléter le rapport et produire un rapport PPT selon le code C et les renseignements pertinents consultés



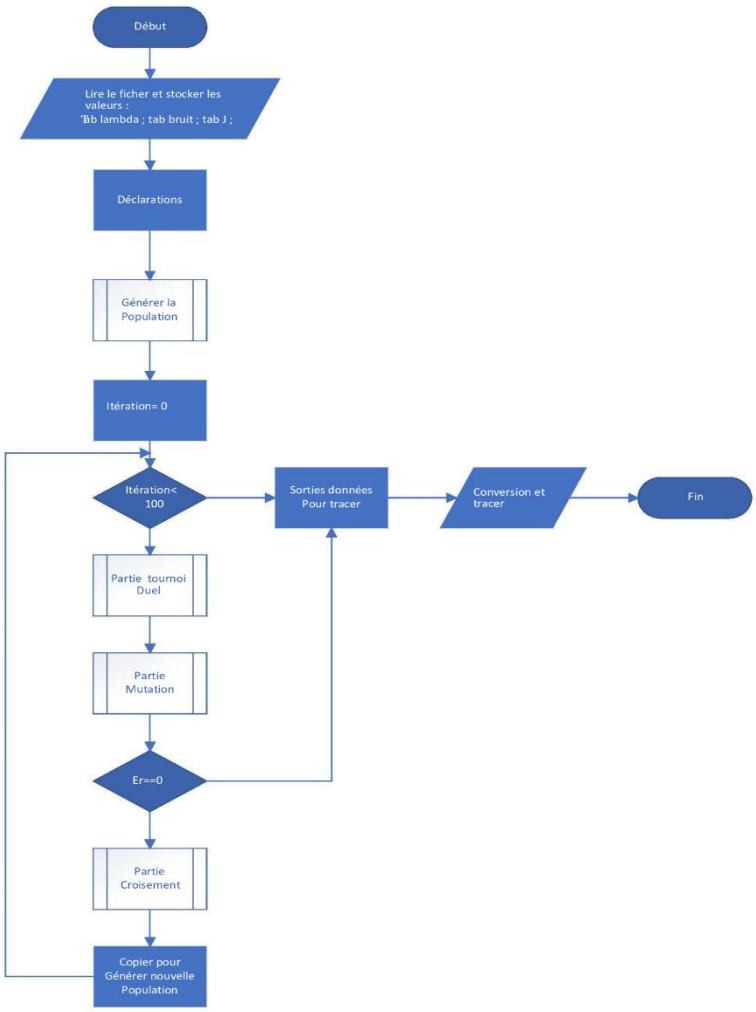
Répartition des travaux :

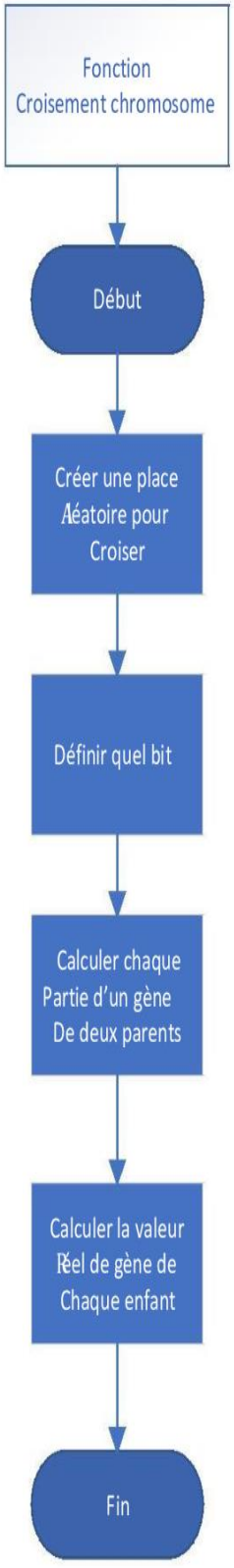
1. Établir l'organigramme de programmation ; Dans le code, lire l'écriture du code de données ; L'écriture du code binaire en conversion décimale ; (**30% par WEI XING**)

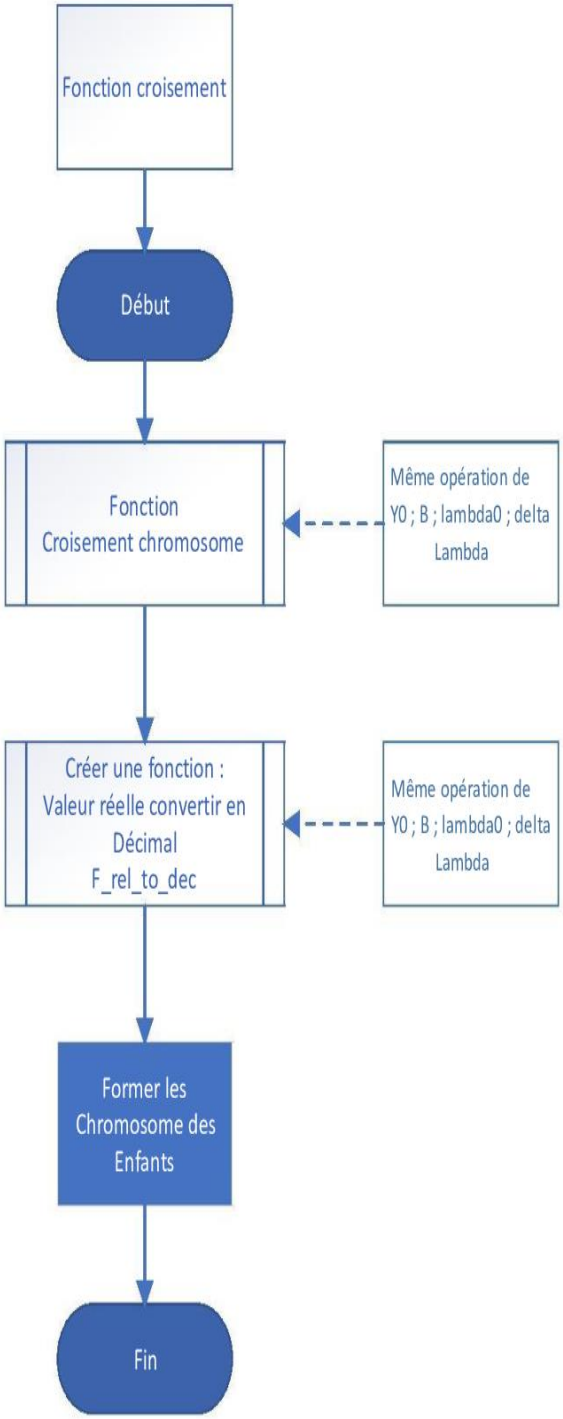
2) Rédaction des quatre sections du rapport (à l'exclusion de la partie analytique des résultats) et synthèse du rapport dans son ensemble ; L'écriture du code décimal de conversion binaire ; Écriture de code des données résultantes finales ; Fiches de taches, dessin du diagramme de Gantt (**30% par XU JINHAO**)

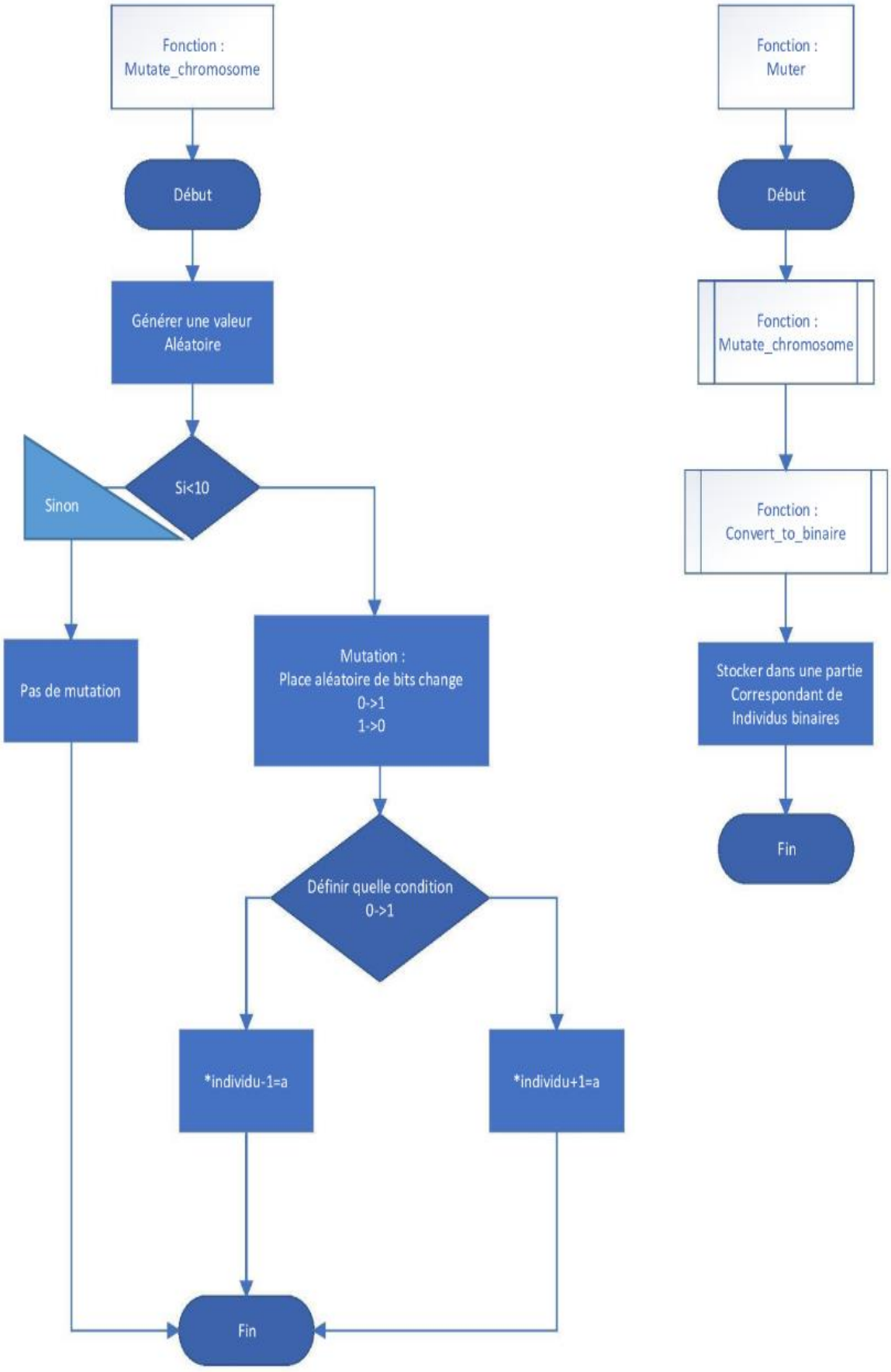
3) Rédaction de la partie analyse critique des résultats du rapport ; Écriture de code dans les sections de sélection, hybridation, variation, nouvelle génération, etc. ; Organisation de conférences (**40% par RAMMAL ZIAD**)

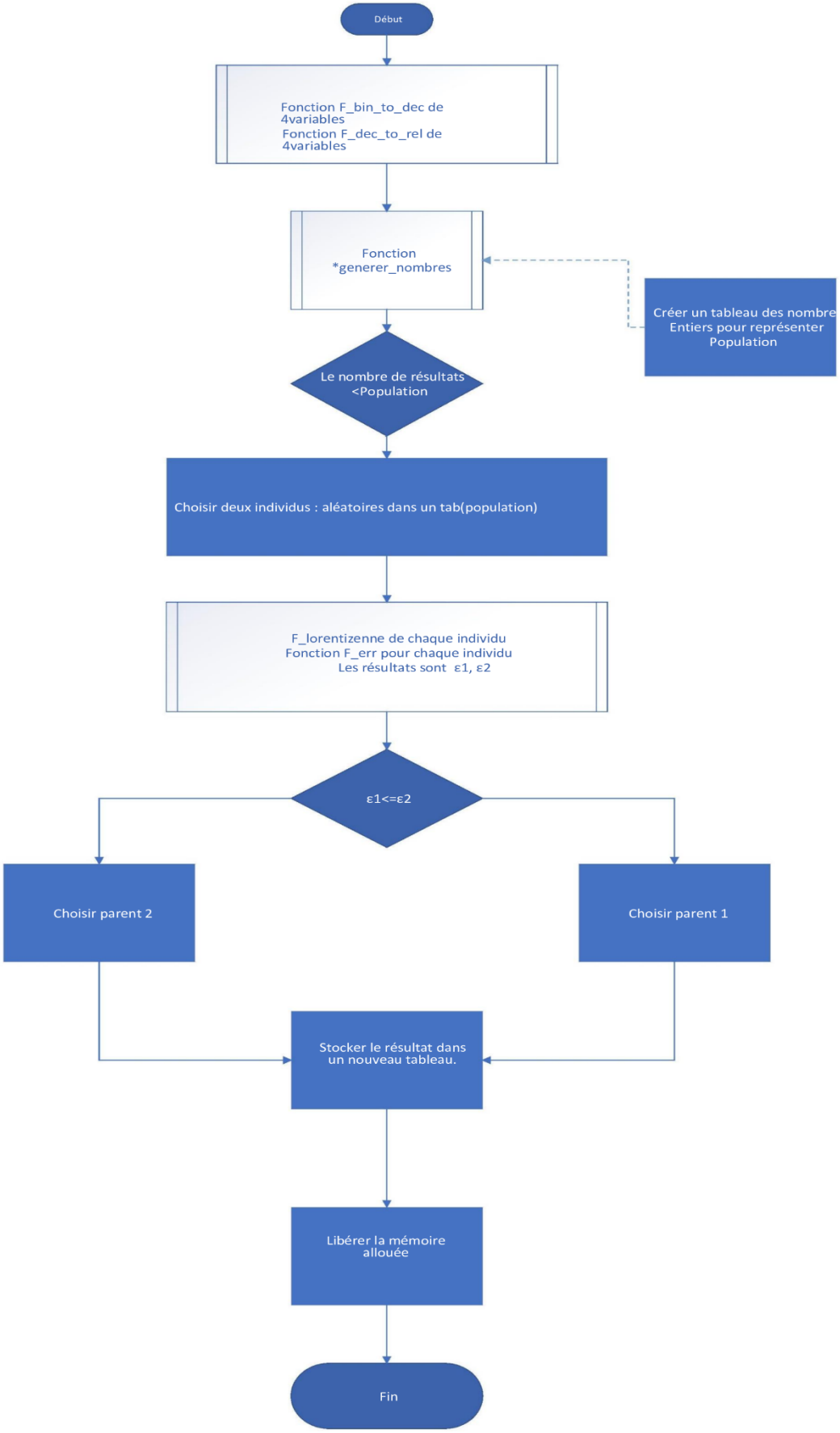
Présentation sous forme de schémas :

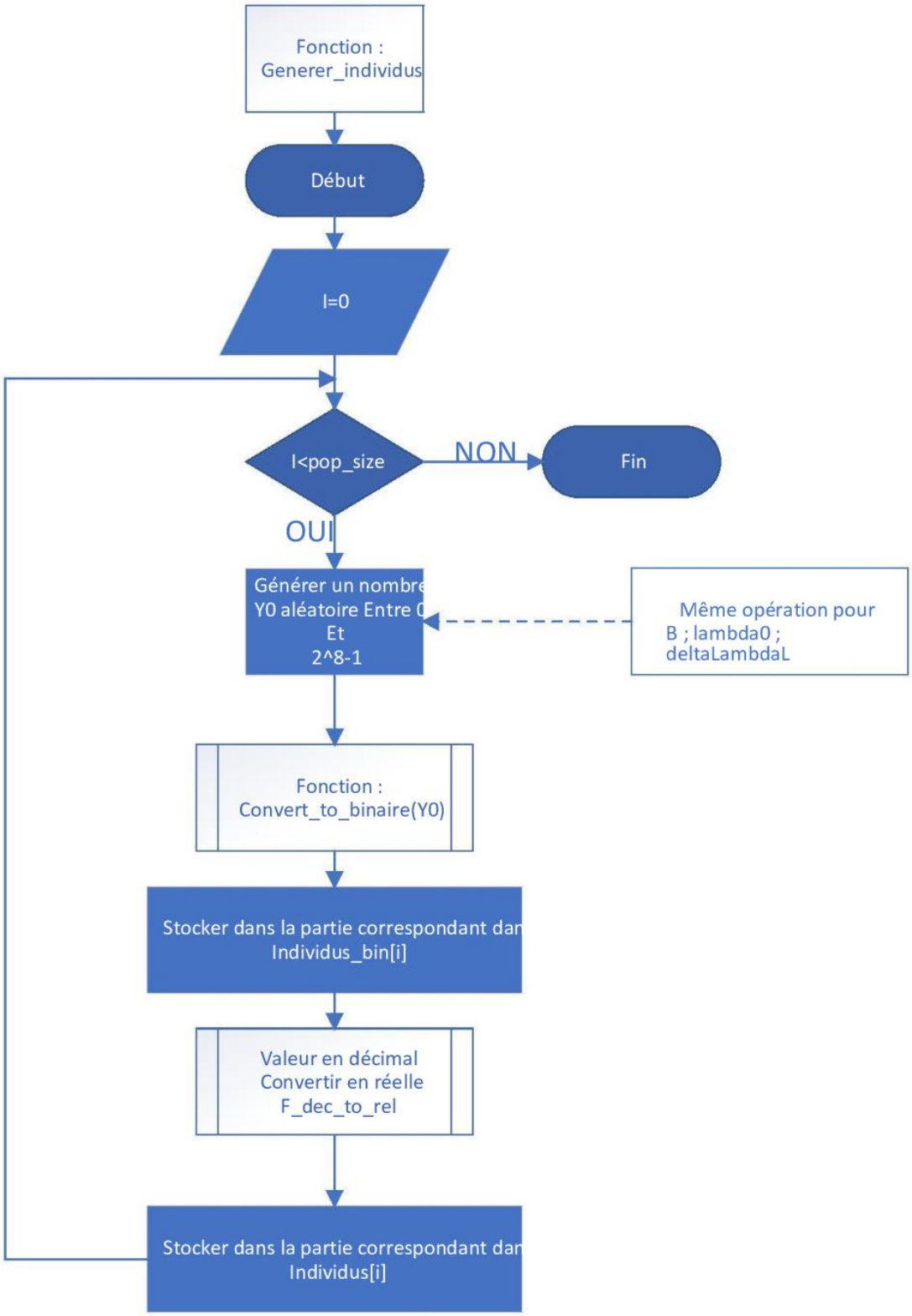


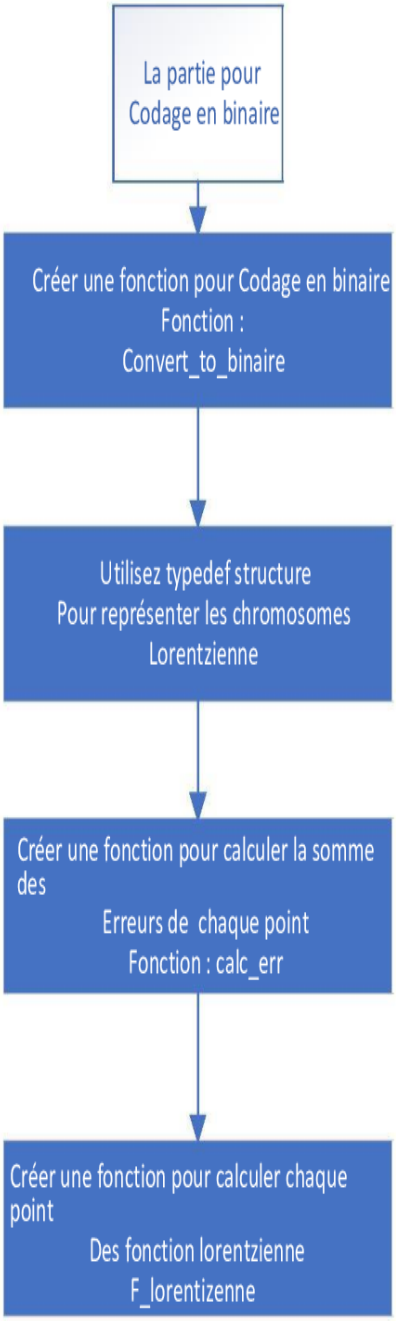

Fonction
Croisement chromosome
Début
Créer une place
Aéatoire pour
Croiser
Définir quel bit
Calculer chaque
Partie d'un gène
De deux parents
Calculer la valeur
Réel de gène de
Chaque enfant
Fin

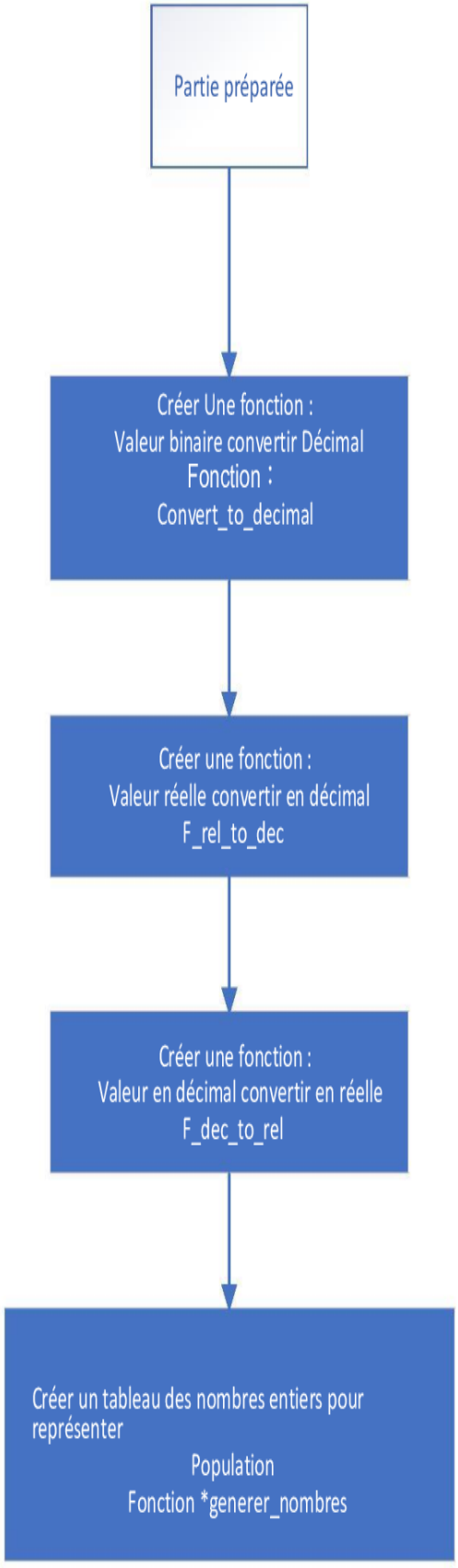

Fonction croisement
Début
Fonction
Croisement chromosome
Même opération de
Y0 ; B ; lambda0 ; delta
Lambda
Créer une fonction :
Valeur réelle convertir en
Décimal
F_rel_to_dec
Même opération de
Y0 ; B ; lambda0 ; delta
Lambda
Former les
Chromosome des
Enfants
Fin


Fonction :
Mutate_chromosome
Début
Générer une valeur
Aléatoire
Si<10
Sinon
Pas de mutation
Mutation :
Place aléatoire de bits change
0->1
1->0
Définir quelle condition
0->1
*individu-1=a
*individu+1=a
Fin
Fonction :
Muter
Début
Fonction :
Mutate_chromosome
Fonction :
Convert_to_binaire
Stocker dans une partie
Correspondant de
Individus binaires
Fin


Début
Fonction F_bin_to_dec de 4variables
Fonction F_dec_to_rel de 4variables
Fonction *generer_nombres
Créer un tableau des nombre Entiers pour représenter Population
Le nombre de résultats <Population
Choisir deux individus : aléatoires dans un tab(population)
F_lorentizenne de chaque individu
Fonction F_err pour chaque individu
Les résultats sont ε1, ε2
ε1<=ε2
Choisir parent 2
Choisir parent 1
Stocker le résultat dans un nouveau tableau.
Libérer la mémoire allouée
Fin


Fonction :
Generer_individus
Début
I=0
I<pop_size
NON
Fin
OUI
Générer un nombre
Y0 aléatoire Entre 0
Et
2^8-1
Même opération pour
B ; lambda0 ;
deltaLambdaL
Fonction :
Convert_to_binaire(Y0)
Stocker dans la partie correspondant dan
Individus_bin[i]
Valeur en décimal
Convertir en réelle
F_dec_to_rel
Stocker dans la partie correspondant dan
Individus[i]


La partie pour
Codage en binaire
Créer une fonction pour Codage en binaire
Fonction :
Convert_to_binaire
Utilisez typedef structure
Pour représenter les chromosomes
Lorentzienne
Créer une fonction pour calculer la somme des
Erreurs de chaque point
Fonction : calc_err
Créer une fonction pour calculer chaque point
Des fonction lorentzienne
F_lorentizenne


Partie préparée
Créer Une fonction :
Valeur binaire convertir Décimal
Fonction :
Convert_to_decimal
Créer une fonction :
Valeur réelle convertir en décimal
F_rel_to_dec
Créer une fonction :
Valeur en décimal convertir en réelle
F_dec_to_rel
Créer un tableau des nombres entiers pour représenter
Population
Fonction *generer_nombres

1. Le codage :

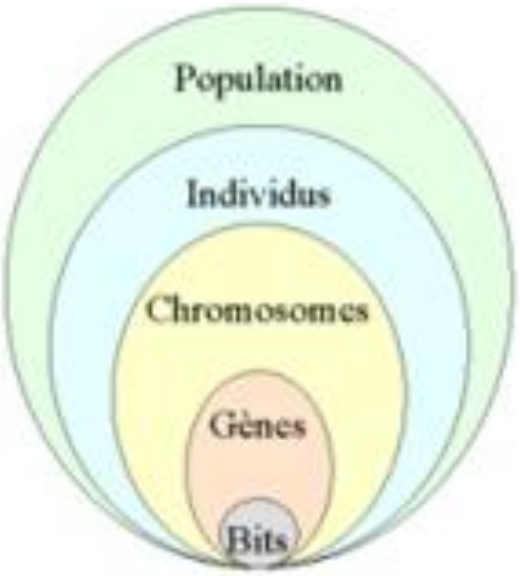
1.1Comment représenter les gènes, les chromosomes, les individus

Le gène : le codage binaire : c'est le plus utilisé. Chaque gène dispose du même alphabet binaire {0, 1} Un gène est alors représenté par un entier long (8 bits)

Les chromosomes : les chromosomes qui sont des suites de gènes sont représentés par des tableaux de gènes. On utilise typedef en langue C.

Les individus : la fonction Lorentzienne représente les individus.

1.2 codage 8 bits entre le maximum et le minimum



2. Les sélections

2.1Comment faire les sélections et les évolutions ?

On utilise La sélection par tournois. Cette méthode est celle avec laquelle on obtient les résultats les plus satisfaisants. Le principe de cette méthode est le suivant : on effectue un tirage avec remise de deux individus de P, et on les fait "combattre". Celui qui a le fitness la plus élevée l'emporte. On répète ce processus n fois de manière à obtenir les n individus de P' qui serviront de parents.

2.2 la sélection tournoi dans cette question

On calcule la somme des erreurs des chaque point entre les valeurs données et les valeurs actuelles. On choisit l'individu avec l'erreur moindre.

3.Les croisements

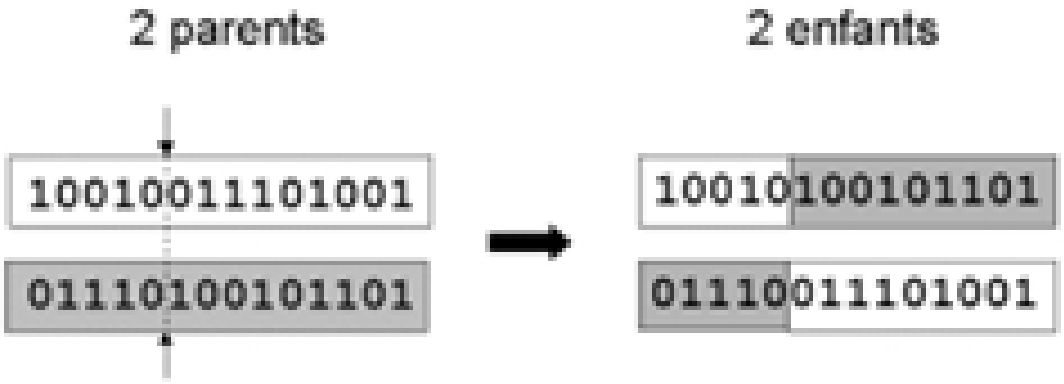


Figure 5: croisement avec un point de crossover

Présentation des résultats obtenus avec une analyse critique de ces derniers :

Le programme nous donne en sortie les valeurs des variables a trouvée.

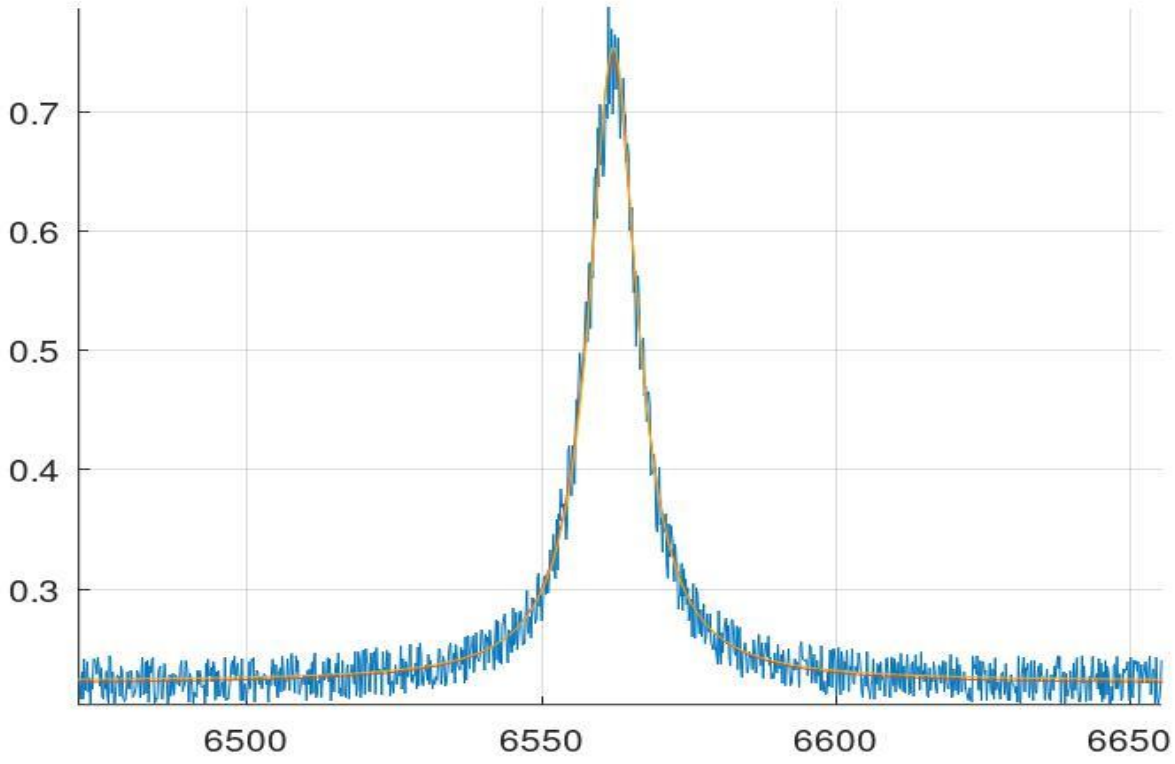
Et crée un dossier contenant les valeurs de lambda et J de la fonction Lorentzienne selon les valeurs calculer par notre algorithme nomme : « Tracee_Finale ».

La sortie de notre programme est la suivante :

Individu : Y0= 0.224431 ; B = 13.198039 ; lambda0 = 6561.596078 ; deltaL = 9.896078

densite = 1098568622474680818728960.000000

Et selon le programme sous octave fournis avec le programme le tracée de cette fonction est la suivante en orange :



Et on peut voir que cette fonction correspond à la fonction bruitée (en bleu) et lisse (en rouge) fournit dans le fichier « Data_Genetique »

Conclusion et inconvénients :

On peut voir que cet algorithme prend du temps si on n'initialise pas nos variables au début de notre programme et on peut voir que ce programme ne donne pas exactement les mêmes valeurs à chaque compilation et c'est dû au nombres aléatoires pris partout dans notre programme.

Et pour résoudre ces problèmes on peut utiliser des différents algorithmes qui peuvent bien résoudre notre problème sans utiliser beaucoup de mémoire.